Traffic Prediction in Bogotá: A Spatio-Temporal Graph Neural Network Approach

A White Paper in collaboration with the Inter-American Development Bank and the Government of Bogotá

Martín Sevilla
martin.sevilla@outlook.com

José María Márquez Blanco
jose.marquez.blanco@gmail.com

January 23, 2026

Abstract

Abstract: Traffic congestion remains one of the most critical challenges for urban planning in Latin America, particularly in high-density metropolitan areas like Bogotá. Accurate forecasting of traffic flow is essential for optimizing traffic management systems and reducing economic and environmental costs. This white paper presents a novel Spatio-Temporal Graph Neural Network (ST-GNN) framework developed in partnership with the Inter-American Development Bank (IDB) and the Government of Bogotá. We review the evolution of traffic prediction models—from classical statistical methods to state-of-the-art deep learning architectures. Specifically, we evaluate the efficacy of Recurrent Graph Convolutional Networks, including **TGCN**, **GConvGRU**, and **GConvLSTM**, as well as attention-based mechanisms like **A3TGCN**. Our proposed model leverages the complex topological structure of Bogotá's road network to deliver precise, real-time traffic predictions.

1 Introduction

Urban mobility is a defining challenge of the 21st century. For cities like Bogotá, which consistently ranks among the most congested urban centers globally, the implications of traffic gridlock extend beyond mere inconvenience; they significantly impact economic productivity, air quality, and the overall quality of life for millions of citizens. In response to these challenges, the Government of Bogotá, in collaboration with the Inter-American Development Bank (IDB), has launched an initiative to modernize the city's traffic management capabilities through Artificial Intelligence (AI) and data-driven decision-making.

The core of this initiative lies in the ability to accurately predict traffic conditions. However, traffic flow is inherently complex, exhibiting dynamic dependencies in both *space* (the connectivity of the road network) and *time* (historical flow patterns). Traditional time-series analysis methods, such as ARIMA, often fail to capture the spatial correlations between adjacent road segments. Conversely, standard Convolutional Neural Networks (CNNs) are limited to grid-like structures and struggle with the irregular topology of a city's road network.

To address these limitations, we employ Spatio-Temporal Graph Neural Networks (ST-GNNs). These models represent the road network as a graph, where road segments are nodes and

their connections (based on traffic flow and proximity) are edges, allowing for the simultaneous extraction of spatial and temporal features.

Recent advancements in this field have produced powerful architectures. The **GConvLSTM** and **GConvGRU** [1] integrate Graph Convolutional Networks (GCNs) with recurrent units, effectively learning sequence patterns on graph structures. Similarly, the Temporal Graph Convolutional Network (**TGCN**) [2] has demonstrated superior performance by combining GCNs with Gated Recurrent Units (GRUs) to capture non-linear traffic dynamics. Furthermore, attention mechanisms have been introduced to weigh the importance of different temporal instances, as seen in the Attention Based Spatial-Temporal Graph Convolutional Network (**A3TGCN**) [3].

This white paper details the deployment of these advanced ST-GNN architectures tailored to the specific constraints and availability of data in Bogotá. By accurately modeling the spatio-temporal dependencies of the city's traffic, we aim to provide actionable insights for the *Secretaría de Movilidad*, facilitating proactive rather than reactive traffic management.

2 Literature Review

Traffic prediction has evolved from simple statistical baselines to complex deep learning frameworks. This section categorizes these approaches into three distinct generations: statistical methods, Euclidean deep learning, and Graph Neural Networks.

2.1 Statistical and Machine Learning Approaches

Early attempts at traffic forecasting relied heavily on statistical theory. The Autoregressive Integrated Moving Average (ARIMA) model and its variants (e.g., Seasonal ARIMA) were widely used due to their mathematical soundness [4]. The ARIMA model can be expressed as:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q} + \epsilon_t, \tag{1}$$

where y_t is the value at time t, c is a constant, ϕ_i are the autoregressive coefficients, θ_i are the moving average coefficients, and ϵ_t is the error term. While effective for stable, short-term predictions on single road segments, these models struggle with the non-linear and stochastic nature of complex traffic networks.

Machine learning methods such as Support Vector Regression (SVR) and k-Nearest Neighbors (k-NN) offered improvements in handling non-linearity but still failed to explicitly model the spatial dependencies between distant roads. For instance, SVR aims to find a function $f(x)$ that minimizes the following objective:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max(0, |y_i - (w \cdot x_i + b)| - \epsilon), \tag{2}$$

where C is a regularization parameter, ϵ is the margin of tolerance, and w and b are the model parameters.

2.2 Euclidean Deep Learning

The advent of Deep Learning introduced Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to the domain. RNNs, particularly Long Short-Term Memory (LSTM) units and Gated Recurrent Units (GRUs), became the standard for capturing temporal dependencies in time-series data. The LSTM cell can be described mathematically as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f), \tag{3}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \tag{4}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C), \tag{5}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{6}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \tag{7}$$

$$h_t = o_t \odot \tanh(C_t), \tag{8}$$

where f_t, i_t, and o_t are the forget, input, and output gates, respectively, C_t is the cell state, and h_t is the hidden state.

Hybrid models, such as CNN-LSTM, attempted to capture spatial features using 2D convolutions [5]. However, CNNs operate on Euclidean domains (grids), making them suboptimal for road networks which are naturally non-Euclidean graphs. Treating a city map as an image often results in a loss of topological information.

2.3 Spatio-Temporal Graph Neural Networks (ST-GNNs)

To effectively model the non-Euclidean structure of road networks, Graph Neural Networks (GNNs) were introduced. Kipf and Welling [6] proposed the Graph Convolutional Network (GCN), which generalizes the convolution operation to graphs using spectral graph theory. The GCN layer can be expressed as:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right), \tag{9}$$

where $\tilde{A} = A + I$ is the adjacency matrix with added self-loops, $\tilde{D}$ is the degree matrix of $\tilde{A}$, $H^{(l)}$ is the input feature matrix at layer l, $W^{(l)}$ is the learnable weight matrix, and σ is an activation function.

In traffic prediction, the goal is to capture both spatial correlations (graph structure) and temporal dynamics (time series). This led to the development of Spatio-Temporal GNNs, which combine GCNs with temporal models like RNNs or attention mechanisms to effectively model the complex dependencies in traffic networks.

2.3.1 Recurrent-Based Approaches

A prominent direction in spatio-temporal graph neural networks involves integrating graph convolution operations into recurrent neural network (RNN) cells. Seo et al. [1] introduced the Graph Convolutional Recurrent Network, which incorporates the Chebyshev spectral graph convolution operator directly into the input-to-state and state-to-state transitions of LSTM and

GRU cells. These models, referred to as *GConvLSTM* and *GConvGRU*, are capable of learning spatial structures that evolve over time. The key idea is to replace the standard matrix multiplication in RNNs with graph convolution operations, enabling the model to capture the topological relationships inherent in the road network. Mathematically, the state update for a GConvGRU can be expressed as:

$$h_t = \sigma\left(W_h \cdot \mathrm{GCN}(A, h_{t-1}) + W_x \cdot x_t + b\right), \tag{10}$$

where h_t is the hidden state at time t, x_t is the input at time t, A is the adjacency matrix of the graph, W_h and W_x are learnable weight matrices, b is the bias term, and σ is a non-linear activation function.

Similarly, Zhao et al. [2] proposed the Temporal Graph Convolutional Network (**TGCN**), which combines a Graph Convolutional Network (GCN) for spatial feature extraction with a GRU for temporal modeling. The GCN captures the topological relationships of the road network, while the GRU models the dynamic changes in traffic data over time. The TGCN can be expressed as:

$$H^{(l+1)} = \sigma\left(AH^{(l)}W^{(l)} + b^{(l)}\right), \tag{11}$$

where $H^{(l)}$ represents the node embeddings at layer l, $W^{(l)}$ is the weight matrix, and $b^{(l)}$ is the bias term. The temporal dynamics are then modeled using a GRU:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \tag{12}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \tag{13}$$

$$\tilde{h}_t = \tanh(W_h x_t + r_t \odot (U_h h_{t-1}) + b_h), \tag{14}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \tag{15}$$

where z_t and r_t are the update and reset gates, respectively, and $\odot$ denotes the element-wise product.

2.3.2 Attention-Based Approaches

While recurrent mechanisms are effective for capturing temporal dependencies, they may struggle with very long-term dependencies and can be computationally expensive. To address these limitations, attention mechanisms have been introduced. Attention mechanisms allow the model to dynamically weigh the importance of different time steps in the historical data, enabling it to focus on the most relevant temporal information.

Zhu et al. [3] proposed the Attention Temporal Graph Convolutional Network (**A3TGCN**), which extends the capabilities of TGCN by incorporating an attention mechanism. This mechanism assigns varying importance to different time points, improving the model's ability to handle scenarios with high variability. The attention mechanism can be expressed as:

$$\alpha_t = \frac{\exp\left(\mathrm{score}(h_t, q)\right)}{\sum_{t'} \exp\left(\mathrm{score}(h_{t'}, q)\right)}, \tag{16}$$

where α_t is the attention weight for time step t, h_t is the hidden state at time t, q is the query vector, and score is a function that computes the relevance of h_t with respect to q. The final

output is then computed as a weighted sum of the hidden states:

$$o = \sum_t \alpha_t h_t. \tag{17}$$

These advanced ST-GNN frameworks provide the theoretical foundation for the predictive model developed for the Bogotá traffic project, enabling the effective modeling of complex spatio-temporal dependencies in urban traffic networks.

3 Data Processing and Infrastructure

The deployment of Spatio-Temporal Graph Neural Networks for traffic forecasting in Bogotá presented unique computational and engineering challenges that required innovative solutions. This section details the comprehensive data processing pipeline developed to handle the scale and complexity of urban traffic data while ensuring efficient training and inference capabilities.

3.1 Feature Engineering and Data Sources

Our approach leverages a diverse array of urban data sources to capture the multifaceted nature of traffic dynamics in Bogotá. The feature engineering pipeline processes multiple heterogeneous datasets into a unified spatial-temporal representation suitable for graph-based learning.

3.1.1 Primary Traffic Features

The core traffic data is derived from Waze's crowdsourced traffic jams dataset, providing real-time insights into congestion patterns across the city's road network. The `JamsProcessor` extracts and spatially assigns traffic features to road segments through a sophisticated geometric matching algorithm:

Traffic data from Waze's crowdsourced traffic jams dataset serves as the primary source for understanding congestion patterns across Bogotá's road network. To integrate this data into the model, a comprehensive processing pipeline was developed. First, a spatial assignment process was implemented, where traffic jams were mapped to specific road segments using a 4-meter buffer around each road's geometry. This approach ensures accurate spatial alignment between the reported jams and the corresponding road segments. Additionally, overlap thresholds were applied to further refine the spatial join process.

To align the temporal granularity of the raw jam data with the model's requirements, the data was aggregated into 15-minute intervals. This temporal resolution was chosen to balance the need for capturing traffic dynamics with the computational efficiency required for large-scale data processing. The aggregated data includes multiple features for each road segment and time window, such as congestion severity (jam level), delay metrics, average speeds, and jam count statistics. These features provide a multi-dimensional representation of traffic conditions, enabling the model to capture the complex dynamics of urban mobility.

Environmental and contextual factors, such as weather conditions, also play a significant role in influencing traffic patterns, particularly in tropical cities like Bogotá. To incorporate these factors, meteorological data from multiple weather stations was processed and integrated into

the model. Key weather variables, including wind direction and speed, humidity, precipitation, and temperature, were collected and assigned to road segments based on their proximity to weather stations. A 15-meter buffer was used to ensure that each road segment received relevant environmental context. Furthermore, the weather data was temporally aligned with the traffic data using the same 15-minute resolution, ensuring consistency in the temporal representation of the dataset.

Static urban characteristics were also considered to provide additional context for traffic generation and flow patterns. Building footprints were extracted from OpenStreetMap and categorized by type, such as residential, commercial, industrial, and public buildings. These classifications help capture the varying traffic generation patterns associated with different land uses. Building density metrics were calculated by counting the number of buildings within 100-meter buffers of each road segment, providing local density indicators. Additionally, population data from Traffic Analysis Zones (ZAT) was spatially joined to road segments, offering demographic context for traffic demand modeling.

Disruptions to normal traffic flow, such as accidents, road closures, and public events, were also accounted for in the data processing pipeline. Traffic incidents were assigned directly to affected road segments using precise spatial matching techniques. Public events, such as concerts, demonstrations, and sporting events, were modeled using a network-based propagation approach. This method allowed the impact of events to spread through connected road segments based on the complexity and scale of the event, providing a more realistic representation of their influence on traffic patterns.

Finally, we implemented `RoadGraphConstructor` to transform Bogotá's road network into a directed graph, which was specifically designed to optimize the data for spatial-temporal learning. Each road segment was represented as a graph node, enriched with static features such as geometric properties (e.g., road length, cardinal direction, and flow directionality), land use context (e.g., building counts by type, population density, and land use composition), and transit infrastructure (e.g., bus stop counts and proximity to public transportation corridors). This comprehensive representation of the road network ensures that the model can effectively capture the unique characteristics of Bogotá's urban environment.

3.1.2 Edge Construction with Legal Traffic Flow

The construction of edges in the road network graph was carefully designed to incorporate the legal traffic flow constraints and spatial proximity relationships. Legal connections, which represent the navigable paths between road segments based on traffic directionality and intersection geometry, were assigned a weight of 1.0. These connections ensure that the graph accurately reflects the permissible movements within the road network, capturing the inherent structure and constraints of urban traffic.

In addition to legal connections, spatial proximity connections were introduced to account for the influence of geographically close but not directly connected road segments. These connections, assigned a lower weight of 0.3, allow the Spatio-Temporal Graph Neural Network (ST-GNN) to learn spatial correlations that may exist between nearby road segments, even in the absence of direct connectivity. This approach enhances the model's ability to capture the com-

plex spatial dependencies present in urban traffic networks while maintaining respect for the legal traffic flow constraints.

3.2 Memory-Efficient Data Storage and Retrieval

The scale and sparsity of urban traffic data posed significant challenges for storage and retrieval. To address these challenges, a memory-efficient data storage and retrieval system was developed, leveraging the Coordinate (COO) sparse matrix format. This format is particularly well-suited for traffic data, which is characterized by a high degree of sparsity, as most road segments do not experience traffic incidents or specific events at any given time. By storing only non-zero feature values, the COO format significantly reduces the memory footprint, often achieving a reduction of 95–99%.

Each data point in the COO format is represented as a tuple containing the node index, time index, feature index, and value. This structure enables rapid retrieval of specific spatiotemporal slices, facilitating efficient data access during model training. Furthermore, the data is sorted by time index, which allows for the use of binary search operations to perform time-range queries efficiently.

For example, consider the following dense matrix representation of traffic data:

	t_1	t_2	t_3
n_1	0	5	0
n_2	0	0	3
n_3	7	0	0

In the COO format, only the non-zero entries are stored along with their corresponding indices:

Node Index	Time Index	Value
n_1	t_2	5
n_2	t_3	3
n_3	t_1	7

As shown, the COO representation significantly reduces the storage requirements by omitting zero entries, making it an ideal choice for handling sparse traffic data.

To further optimize storage and retrieval, the processed sparse data is stored in the HDF5 format. This format offers several advantages, including lazy loading, which ensures that only the required data slices are loaded into memory during training. This capability is particularly important for processing datasets that exceed the available RAM, as it enables efficient training without the need for extensive hardware resources. Additionally, HDF5's built-in compression reduces disk storage requirements while maintaining rapid access times. Pre-computed time-based indices are also cached, enabling constant-time access to temporal windows during training.

These innovations in data storage and retrieval were critical for managing the scale and complexity of the urban traffic data. The combination of sparse matrix representation and HDF5 storage architecture ensured that the training process remained computationally feasible, even on commodity hardware, while preserving the rich spatiotemporal relationships essential

for accurate traffic forecasting.

3.3 Intelligent Batching Strategy

A critical innovation in our approach is the geographic coherence-preserving batching strategy, essential for effective GNN training on large urban networks.

3.3.1 Seed Node Selection and Neighborhood Expansion

To ensure effective training of the Spatio-Temporal Graph Neural Network (ST-GNN) on Bogotá's road network, a systematic seed node selection strategy was employed. Instead of relying on random sampling, which could lead to uneven representation of the road network, seed nodes were carefully chosen to optimize coverage and ensure a balanced representation of the urban zones. The selection process was designed to minimize overlap between batches while maximizing network coverage, ensuring that every road segment was included in at least one batch during training. Additionally, the spatial distribution of seed nodes was carefully managed to avoid clustering in high-density areas, thereby maintaining a balanced representation of both central and peripheral urban zones.

Once the seed nodes were selected, each batch was constructed by expanding from a seed node using a Breadth-First Search (BFS) algorithm. This approach ensured that each batch contained geographically contiguous road segments, which is critical for preserving local spatial relationships that are essential for effective graph neural network learning. The size of each batch was carefully controlled, typically limited to 500 nodes, to fit within the memory constraints of the GPU while maintaining sufficient spatial context for the model. To further enhance the training process, seed nodes were dynamically reassigned during training, ensuring that all areas of the city were represented across multiple training epochs. This systematic batching strategy was crucial for maintaining the spatial correlations inherent in the road network, as random batching would have disrupted these relationships and significantly degraded the model's learning effectiveness.

3.4 Missing Data Imputation Strategy

Urban traffic data often contains missing values due to various factors such as sensor failures, data transmission issues, and inconsistent data source coverage. To address this challenge, an intelligent imputation strategy was developed, leveraging the graph structure of the road network to handle missing data effectively. The imputation process primarily relied on neighbor-based methods, where missing values for critical features such as speed and delay were estimated using aggregated values from spatially neighboring road segments. This approach ensured that the imputed values were contextually relevant and reflective of the local traffic conditions.

Different imputation methods were applied based on the characteristics of the missing features. For instance, temporal interpolation techniques were employed to estimate missing weather data, while spatial averaging was used for traffic metrics. This feature-specific approach allowed the system to handle the diverse nature of the data more effectively. Furthermore, the imputation strategy was designed to preserve data sparsity where appropriate, rather than aggressively

filling in all missing values. This approach allowed the model to learn from the natural distribution patterns of the data, ensuring that the imputation process did not introduce artificial biases or distortions.

3.5 Normalization and Scale Handling

The diverse range of features in the dataset, which included binary event indicators, continuous speed measurements, and other heterogeneous data types, necessitated a robust normalization strategy to ensure numerical stability and preserve the relative importance of each feature. To address this, feature-wise normalization was applied, with each feature being independently normalized using dataset-specific statistics. This approach ensured that all features were scaled appropriately, facilitating effective learning by the model.

To maintain consistency between the training and inference phases, normalization constants were computed globally across the entire temporal range of the dataset. This ensured that the normalized values were comparable across different time periods, avoiding potential discrepancies that could arise from using separate normalization parameters for training and inference. To minimize memory overhead, normalization was applied on-the-fly during data loading using precomputed tensors. This approach allowed the system to handle large datasets efficiently while maintaining the integrity of the normalized data.

By implementing these strategies for seed node selection, missing data imputation, and normalization, the data processing pipeline was able to effectively prepare the urban traffic data for training sophisticated ST-GNN models. These methods ensured that the model could learn from the complex spatio-temporal relationships inherent in Bogotá's road network, ultimately enhancing the accuracy and robustness of the traffic forecasting system.

This comprehensive data processing pipeline enables the training of sophisticated ST-GNN models on city-scale traffic data while maintaining computational efficiency and preserving the rich spatial-temporal relationships essential for accurate traffic forecasting.

4 Model Architecture and Training Infrastructure

The implementation of Spatio-Temporal Graph Neural Networks for Bogotá's traffic forecasting system required careful consideration of model architecture design, scalable training infrastructure, and optimization strategies tailored to the unique challenges of urban traffic prediction.

4.1 Unified ST-GNN Architecture Framework

Our system implements a unified framework supporting multiple state-of-the-art ST-GNN architectures, allowing for empirical comparison and selection of the most suitable model for Bogotá's traffic patterns.

4.1.1 Model Architecture Components

The proposed framework is composed of three integral components that collectively enable the processing of multi-modal urban data. The first component is the static feature encoder, which

utilizes a multilayer perceptron to transform static road features, such as geometry, land use, and population data, into dense embeddings. This encoder effectively maps high-dimensional static features into a 32-dimensional representation, ensuring that each road segment is enriched with consistent geometric and demographic context.

The second component is the temporal graph convolution module, which serves as the core of the Spatio-Temporal Graph Neural Network (ST-GNN). This module processes historical traffic patterns by employing graph-aware recurrent cells. By concatenating dynamic features with the static embeddings generated by the first component, the temporal graph convolution module is able to process temporal sequences while preserving the spatial relationships inherent in the road network graph. This enables the model to capture the complex spatio-temporal dependencies that characterize urban traffic dynamics.

The third component is the future feature integration module, which is designed to incorporate known future features, such as planned events and weather forecasts, into the prediction process. This module employs a separate recurrent architecture to process these external factors over the forecast horizon, thereby enhancing the model's ability to predict future traffic conditions with greater accuracy.

4.1.2 Supported ST-GNN Variants

The framework is designed to support a variety of state-of-the-art Spatio-Temporal Graph Neural Network (ST-GNN) architectures, each tailored to address specific challenges in traffic forecasting. The first variant, GConvGRU, integrates Graph Convolutional Networks (GCNs) with Gated Recurrent Units (GRUs). This architecture is particularly well-suited for capturing short to medium-term traffic dependencies due to its efficient memory usage and rapid training convergence.

The second variant, Temporal Graph Convolutional Network (TGCN), combines GCN-based spatial processing with GRU-based temporal modeling in a streamlined architecture. TGCN strikes an effective balance between computational efficiency and predictive accuracy, making it a robust choice for modeling urban traffic patterns.

The third variant, GConvLSTM, extends the capabilities of GCNs by incorporating Long Short-Term Memory (LSTM) cells. This architecture excels in modeling longer-term dependencies, such as weekly and seasonal traffic patterns, although it comes with increased computational complexity.

The fourth and final variant, Attention Temporal Graph Convolutional Network (A3TGCN), enhances the TGCN architecture by introducing temporal attention mechanisms. These mechanisms dynamically assign weights to historical time steps, allowing the model to prioritize the most relevant temporal information. This makes A3TGCN particularly effective for handling irregular traffic events and varying pattern importance across different temporal contexts.

4.1.3 Multi-Target Prediction Architecture

A distinguishing feature of the proposed framework is its capability for multi-target prediction, which allows for the simultaneous forecasting of multiple traffic metrics. The primary target of the model is the jam level, which quantifies congestion severity on a scale from 0 to 5 and serves

as the core variable for assessing traffic conditions. In addition to the jam level, the model also predicts the mean jam velocity, which provides complementary insights into the characteristics of traffic flow.

The framework employs a joint optimization strategy to predict both targets simultaneously. By leveraging a shared representation, the model is able to learn correlated patterns between the jam level and mean jam velocity, thereby improving the robustness and accuracy of its predictions. This multi-target approach ensures that the model captures a more comprehensive understanding of traffic dynamics, which is critical for effective traffic management and planning in complex urban environments.

4.2 Scalable Training Infrastructure

Training ST-GNN models on city-scale data requires sophisticated infrastructure capable of handling the computational and memory demands of large graph neural networks.

4.2.1 AWS SageMaker Integration

The training infrastructure for the Spatio-Temporal Graph Neural Network (ST-GNN) models is built on AWS SageMaker, which provides a scalable and managed environment for training. The system utilizes GPU-optimized instances, specifically the ml.g4dn.8xlarge configuration. These instances are equipped with 32 vCPUs, 128 GB of RAM, and NVIDIA T4 GPUs with 16 GB of VRAM, ensuring efficient processing of large graph batches. This hardware configuration is particularly suited for the computational demands of large-scale graph neural network training.

To ensure reproducibility and consistency across different infrastructure configurations, the training environment is containerized using custom Docker containers. These containers include all necessary dependencies, such as PyTorch, PyTorch Geometric, and other specialized graph neural network libraries. This approach simplifies the deployment process and ensures that the training environment remains consistent across different runs and setups.

Additionally, the training pipeline integrates with the AWS Elastic File System (EFS), which provides high-throughput and low-latency access to preprocessed datasets. This integration eliminates the need for local storage constraints by enabling efficient data streaming during training. The use of EFS ensures that the training process can handle large datasets without being limited by the storage capacity of individual instances.

4.2.2 Advanced Memory Management

Given the memory-intensive nature of large-scale graph processing, the training system incorporates several advanced memory optimization techniques. One such technique is dynamic graph batching, where edge indices are constructed dynamically for each batch. This approach allows the creation of batched graph representations that maximize GPU utilization while adhering to memory constraints.

Another critical optimization is gradient accumulation, which enables the use of large effective batch sizes by accumulating gradients across multiple mini-batches. This technique is particularly useful for achieving stable training when GPU memory is limited. Furthermore, the system

employs Automatic Mixed Precision (AMP) training with GradScaler. This method reduces memory usage by 40–50% by performing computations in lower precision while maintaining numerical stability through dynamic loss scaling. These memory management strategies are essential for ensuring that the training process remains efficient and scalable, even when dealing with the extensive data and computational requirements of urban traffic prediction.

4.3 Training Strategy and Optimization

The training process for the ST-GNN models is designed to address the unique challenges of urban traffic prediction. A key aspect of the training strategy is the use of temporal data splitting. Unlike traditional random data splitting, which can lead to unrealistic evaluation scenarios, this approach employs a chronological division of the dataset. Specifically, the training data comprises the first 80% of the temporal sequence, while the most recent 20% is reserved for validation. This method ensures that the evaluation process mimics real-world deployment scenarios, where future data is not available during model training. By strictly preventing future information from influencing past predictions, the temporal splitting approach ensures that the model's performance is evaluated in a realistic and unbiased manner.

The loss function used for training the ST-GNN models is carefully designed to address the specific challenges associated with traffic congestion prediction. The primary component of the loss function is the Mean Squared Error (MSE), which provides the main optimization signal for both jam level and velocity predictions. To enhance the model's ability to predict critical traffic conditions, additional penalties are incorporated into the loss function. A Mean Absolute Error (MAE) penalty is applied for jam levels above a threshold (≥ 2), ensuring that the model accurately captures congestion events that are crucial for traffic management. Furthermore, an enhanced penalty is introduced for severe congestion levels (≥ 4), reflecting the significant impact of extreme traffic disruptions on urban mobility. To account for the operational consequences of under-predicting traffic issues, the loss function also includes a directional loss component. This asymmetric penalty favors the over-prediction of congestion levels, as it is generally more acceptable to overestimate traffic issues than to underestimate them.

The composite loss function is mathematically expressed as:

$$L = L_{MSE} + \lambda_1 L_{MAE}^{high} + \lambda_2 L_{MAE}^{severe} + \lambda_3 L_{directional}, \tag{18}$$

where $\lambda_1 = 0.5$, $\lambda_2 = 0.5$, and $\lambda_3 = 0.3$ are empirically determined weighting factors. These weights are carefully tuned to balance the contributions of each loss component, ensuring that the model effectively learns to predict both typical and extreme traffic conditions.

To ensure comprehensive spatial coverage during training, a systematic node rotation strategy is employed. This strategy involves selecting different seed nodes for batch construction in each training epoch, ensuring that all road segments are included in the training process over multiple epochs. The seed selection process is optimized to minimize overlap between batches while maximizing network coverage. Typically, 8-12 distinct seed nodes are required to cover the entire road network of Bogotá. The geographic distribution of seed nodes is carefully managed to prevent overrepresentation of dense urban areas and to ensure adequate coverage of periph-

eral regions. This systematic approach to node rotation and batch construction is critical for maintaining the spatial correlations inherent in the road network, which are essential for effective graph neural network learning.

The training configuration is designed to balance computational efficiency with model performance. Input sequences of 150 timesteps (equivalent to 37.5 hours) are used, with output horizons of 50 timesteps (12.5 hours). This configuration provides sufficient historical context for the model to learn from while maintaining practical forecast utility. The training process employs the Adam optimizer with a learning rate of 0.001, a gradient clipping norm of 1.0, and a batch size of 16, with each batch containing approximately 500 nodes. To minimize I/O bottlenecks during GPU computation, the data loading process is optimized using multi-worker data loading with four workers, persistent workers, and prefetching with a factor of two. Comprehensive regularization techniques, including tensor validation, NaN/infinity detection, and statistical monitoring, are implemented to prevent training instabilities and ensure the robustness of the model.

The training system also incorporates extensive monitoring and validation capabilities to ensure the quality and reliability of the trained models. Real-time metrics, including training and validation losses, component-wise loss breakdowns, and computational timing metrics, are logged and visualized to enable continuous monitoring of the training process. Integration with AWS SageMaker's metric definitions allows for automatic capture and visualization of training progress through AWS CloudWatch, facilitating remote monitoring of extended training runs. Regular model checkpointing is performed to preserve the model state, enabling training recovery in case of interruptions and allowing for model selection based on validation performance.

By combining advanced memory management techniques, a robust training strategy, and comprehensive monitoring capabilities, the training infrastructure is well-equipped to handle the challenges of training sophisticated ST-GNN models on large-scale urban traffic data. This ensures that the resulting models are both accurate and efficient, making them suitable for deployment in real-world traffic management systems.

References

[1] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, "Structured sequence modeling with graph convolutional recurrent networks," 2016.

[2] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, pp. 3848–3858, Sept. 2020.

[3] J. Zhu, Y. Song, L. Zhao, and H. Li, "A3t-gcn: Attention temporal graph convolutional network for traffic forecasting," 2020.

[4] B. Williams and L. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal arima process: Theoretical basis and empirical results," *Journal of Transportation Engineering*, vol. 129, pp. 664–672, 11 2003.

[5] H. Yao, F. Wu, J. Ke, X. Tang, Y. Jia, S. Lu, P. Gong, J. Ye, and Z. Li, "Deep multi-view spatial-temporal network for taxi demand prediction," 2018.

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017.